Faculty of Computing

Year 1 Semester 2 (2025)

SE1020 – Object Oriented Programming Lab Sheet 06

Question 1- Inheritance and Polymorphism (from Tutorial 03)

Implement an **Online Course Enrollment** System that manages different types of courses and calculates the total course fees for students. The platform offers both Regular Courses and Premium Courses. Premium courses include additional certification services, while regular courses do not.

a) Create a class called **Course** with courseID (String), courseName (String), and durationWeeks (int) as attributes.

b) Create a default constructor and a parameterized constructor to assign all details of a Course.

c) Implement a method called *displayCourseDetails()* to print course information.

d) Implement method overloading as below.

   i. Implement *calculateTotalFee()* with no parameters to return 10000.00as the basic course fee.

   ii. Overload *cacalculateTotalFee* method with one double parameter as discountPercentage to calculate the total fee using the discount value.
   [Hint: call the calculateTotalFee() method get the basic course fee]

e) Create a subclass called **RegularCourse** that inherits from Course class and do followings.

   i. Add an additional attribute hasLiveSessions (boolean) to indicate whether live sessions are included.

   ii. Implement a default constructor and a parameterized constructor constructor to assign all details of a Regular Course (including coursed, courseName and durationWeeks).

   iii. Override the calculateTotalFee() method to compute the total fee as;

$$Total\ Fee = 100 \times durationWeeks$$

   iv. Override the *displayCourseDetails()* method to display all the details including the total fee and live session status.

f) Create another subclass called **PremiumCourse** that inherits from Course and do followings.

   i. Add an additional attribute called includeCertification (boolean).
   ii. Implement a default constructor and a parameterized constructor constructor to assign all details of a Premium Course (including coursed, courseName and durationWeeks).
   iii. Override the calculateTotalFee() method to compute the total fee as;

Total Fee = (150 × durationWeeks) + (Certification Fee of $50 if includeCertification is true)

   iv. Override the *displayCourseDetails()* method to include the total fee and certification status.

g) Implement the main method as follows;

   i. Create one object from **RegularCourse** using the default constructor.
   ii. Create one **PremiumCourse** object using parameterized constructors.
   iii. Call both overloaded versions of calculateTotalFee() for the objects.
   iv. Display all course details using the overridden displayCourseDetails() method (refer to the output given below).

   Sample Output:

```
=== Regular Course (Default Constructor) ===
Course ID: N/A
Course Name: N/A
Duration: 0 weeks
Live Sessions Included: false
Total Fee: $0.0
Basic Fee: $0.0
Discounted Fee (10%): $0.0

=== Premium Course (Parameterized Constructor) ===
Course ID: PC202
Course Name: Machine Learning
Duration: 6 weeks
Certification Included: true
Total Fee: $950.0
Basic Fee: $950.0
Discounted Fee (20%): $760.0
```

Question 02

Create a Fitness Center to develop a **Membership Management System** that tracks different types of memberships and calculates total fees based on the services offered. The center offers both **Basic** Memberships and **Premium** Memberships.

- Basic Membership includes gym access only.
- Premium Membership includes gym access, personal training, and spa facilities.

The system should allow flexible membership options, fee calculations with discounts, and provide detailed membership information.

a) Create a class called **Membership** with the attributes membershipID (String), memberName (String), durationMonths (int). Implement the following in the Membership class:

   i. A default constructor and a parameterized constructor to assign all membership details.

   ii. A method *displayMembershipDetails()* to print basic membership information

b) Implement method overloading in the Membership class as below:

   i. *calculateFee()* - Returns 5000.00 as the base membership fee.

   ii. *calculateFee(double discountPercentage)* - Applies a discount to the base fee. *[Hint: Call calculateFee() to get the base fee]*

c) Create a subclass called **BasicMembership** that inherits from Membership. Add an additional attribute hasGymAccess (boolean) to indicate if gym access is included.

   i. Implement a default constructor and a parameterized constructor to assign all details of a Basic Membership.

   ii. Override the *calculateFee()* method to compute the total fee as;

   $$\textbf{Total Fee} = \textbf{200} \times \textbf{durationMonths}$$

   iii. Override the *displayMembershipDetails()* method to include the total fee and gym access status.

d) Create another subclass called **PremiumMembership** that inherits from Membership. Add an additional attribute includePersonalTraining (boolean).

   i. Implement a default constructor and a parameterized constructor to assign all details of a Premium Membership.

   ii. Override the *calculateFee()* method to compute the total fee as;
   Total Fee = (300 × durationMonths) + (Personal Training Fee of $100 if includePersonalTraining is true)

iii. Override the *displayMembershipDetails()* method to include the total fee and personal training status.

e) Create another class called **MembershipApp** with main method and perform below tasks.

i. Create one object from **BasicMembership** using the default constructor.

ii. Create one object from **PremiumMembership** using the parameterized constructor.

iii. Call both overloaded versions of *calculateFee()* for the objects.

iv. Display all membership details using the overridden *displayMembershipDetails()* method.

Question 03

Develop a **Shape Area Calculation System** that computes the area of different geometric shapes. The system should support rectangles and triangles, both of which are special types of shapes.

- A Rectangle has a length and width.
- A Triangle has a base and height.

a) Create a class called **Shape** with the attribute shapeName (String). Implement a default constructor and a parameterized constructor to assign the shape name. Implement a method *displayShapeDetails()* to print the shape name.

b) Implement method overloading in the Shape class by defining calculateArea() with no parameters to return 0.0 as the default area and an overloaded method calculateArea(double factor), which scales the area by multiplying the default area with the given factor.

c) Create a subclass called Rectangle that inherits from Shape. Add two additional attributes: length (double) and width (double). Implement a default constructor and a parameterized constructor to assign all details of a Rectangle. Override the calculateArea() method to compute the area as Area = length $\times$ width and override the displayShapeDetails() method to include the area.

d) Create another subclass called Triangle that inherits from Shape. Add two additional attributes: base (double) and height (double). Implement a default constructor and a parameterized constructor to assign all details of a Triangle. Override the calculateArea() method to compute the area as Area = (base $\times$ height) / 2 and override the displayShapeDetails() method to include the area.

e) Implement the main() method to create one object from Rectangle using the default constructor and one object from Triangle using the parameterized constructor. Call both overloaded versions of calculateArea() for the objects and display all shape details using the overridden displayShapeDetails() method.

Question 04

Please submit your answer to the question below to Git. When committing the file, ensure that you include a meaningful commit message (e.g., "Lab06 - Question04").

Implement a **Vehicle Speed Calculation System** that computes the speed of different types of vehicles. The system should support cars and bicycles, both of which are specific types of vehicles.

- A Car has an engine capacity (CC) and a top speed (km/h).
- A Bicycle has a gear count and a maximum pedaling speed (km/h).

a) Create a class called Vehicle with the attribute vehicleName (String). Implement a method displayVehicleDetails() to print the vehicle name.

b) Implement method overloading in the Vehicle class by defining calculateSpeed() with no parameters to return 0.0 as the default speed and an overloaded method calculateSpeed(double factor), which scales the speed by multiplying the default speed with the given factor.

c) Create a subclass called Car that inherits from Vehicle. Add two additional attributes: engineCapacity (int) and topSpeed (double). Override the calculateSpeed() method to return the car's top speed, and override the displayVehicleDetails() method to include the speed and engine capacity.

d) Create another subclass called Bicycle that inherits from Vehicle. Add two additional attributes: gearCount (int) and maxPedalSpeed (double). Override the calculateSpeed() method to return the bicycle's maximum pedaling speed, and override the displayVehicleDetails() method to include the speed and gear count.

e) Implement the main() method to create one object from Car using the default constructor and one object from Bicycle using the parameterized constructor. Call both overloaded versions of calculateSpeed() for the objects and display all vehicle details using the overridden displayVehicleDetails() method.